AJAY.I - 71812401007 · AGENTIC AI COGNITION · MODULE 2 — PLANNING AND MEMORY ARCHITECTURES NOTES

1. TEST-TIME COMPUTE

1.1 What is Test-Time Compute?

Simple explanation:

Test-Time Compute is nothing but giving an AI model more time and computational resources to think about a problem before providing its final answer.

For example, imagine asking an AI:

"Solve this complicated mathematical problem."

Instead of immediately giving an answer, the AI can:

1. Analyze the problem.
2. Try different approaches.
3. Calculate possible solutions.
4. Verify the calculations.
5. Select the correct answer.

The additional computation used during this process is called Test-Time Compute.

Technical explanation:

Test-Time Compute refers to the computational resources allocated to an AI model during inference to improve its reasoning, problem-solving and output quality.

It allows models to perform additional reasoning, generate multiple candidate solutions, explore alternatives and verify results without necessarily changing the model's trained parameters.

1.2 Why is Test-Time Compute needed?

A normal language model may generate an answer immediately.

However, difficult problems require more reasoning.

For example:

- Simple question: What is 5 + 5?
- Complex question: Design an optimized algorithm for processing one million records.

The second problem requires more computation, analysis and verification.

Test-Time Compute allows the model to spend additional resources on such difficult tasks.

1.3 Working of Test-Time Compute

User Question → **AI Model** — Understands the problem → **Allocate Additional Compute** — Reasoning, searching and verification → **Evaluate Possible Solutions** → **Final Answer**

1.4 Techniques used in Test-Time Compute

1. Extended Reasoning

Simple explanation: The AI spends more time thinking through the problem.

Technical explanation: The model generates additional intermediate reasoning steps to solve complex tasks.

Example: Solving a mathematical problem using multiple logical steps.

2. Multiple Candidate Generation

Simple explanation: The AI tries different answers instead of depending on the first answer.

Technical explanation: The model generates multiple candidate solutions and compares them using evaluation mechanisms.

Example: A navigation agent explores three different routes and selects the most suitable one.

3. Verification

Simple explanation: The AI checks whether its answer is correct.

Technical explanation: The model evaluates generated outputs using test cases, external tools, rules or verification mechanisms.

Example: A coding agent executes its generated Python program and checks the output.

4. Search-Based Reasoning

Simple explanation: The AI explores different possible paths to find a suitable solution.

Technical explanation: Search algorithms explore candidate states or reasoning paths to identify promising solutions.

Example: An AI agent explores different chess moves before selecting one.

1.5 Advantages

- Improves the opportunity for accurate answers.
- Supports complex reasoning.
- Helps explore alternative solutions.
- Enables solution verification.
- Useful for mathematical and programming tasks.

1.6 Limitations

- Requires additional computational resources.
- Increases processing time.
- May increase inference cost.
- More computation does not guarantee correctness.

Important: Test-Time Compute happens during inference, whereas model training happens before deployment to learn the model's parameters.

2. GENERATE–VERIFY–REFINE PARADIGM

2.1 What is Generate–Verify–Refine?

Simple explanation:

Generate–Verify–Refine is nothing but a method where an AI first creates an answer, checks whether it is correct and improves it if necessary.

Imagine an AI coding agent.

- Generate: It writes Python code.
- Verify: It runs the code and checks for errors.
- Refine: It corrects the errors and produces improved code.

This three-stage process is called Generate–Verify–Refine.

Technical explanation:

Generate–Verify–Refine is an iterative problem-solving paradigm in which an AI system generates an initial solution, evaluates it against correctness criteria and modifies it based on verification feedback.

The process can repeat until the solution satisfies the required conditions or a stopping criterion is reached.

2.2 Three Main Stages

1. Generate

Simple explanation: Create the first solution.

Technical explanation: The language model generates an initial response, program, plan or candidate solution based on the given input.

Example: Generate Python code to calculate the average of five numbers.

2. Verify

Simple explanation: Check whether the generated solution is correct.

Technical explanation: The system evaluates the generated solution using test cases, execution results, validation rules or external feedback.

Example: Execute the Python program and check whether the average is calculated correctly.

3. Refine

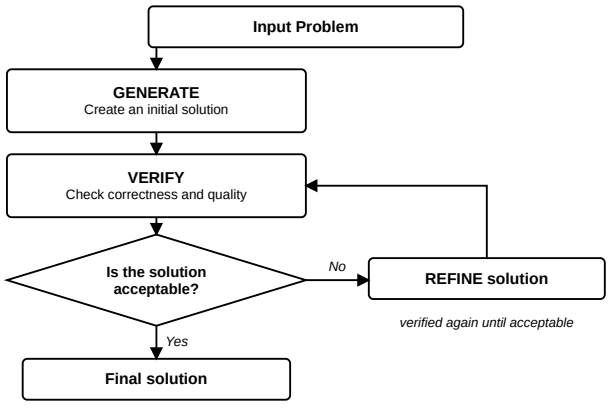
Simple explanation: Correct the mistakes and improve the solution.

Technical explanation: The model uses verification feedback to revise the initial output and generate an improved solution.

Example: Correct a calculation error in the Python code.

2.3 Architecture

DIAGRAM



Generate–Verify–Refine architecture

The refined solution is verified again until it satisfies the required conditions.

2.4 Python Example

```python
# Generate
numbers = [10, 20, 30, 40]
average = sum(numbers) / len(numbers)

# Verify
expected = 25
if average == expected:
    print("Solution verified")
else:
    print("Refining solution")

# Final result
print("Average:", average)
```

Output:

```
Solution verified
Average: 25.0
```

Applications

- Code generation and debugging.
- Mathematical problem-solving.
- Report generation.
- Automated testing.
- AI-based decision-making.

Advantages

- Reduces avoidable mistakes.
- Improves output quality.
- Supports iterative improvement.
- Provides a systematic verification process.

Important difference: Test-Time Compute is the additional computation used during inference. Generate–Verify–Refine is one approach that can use that computation.

3. VECTOR STORES

3.1 What is a Vector Store?

Simple explanation:

A Vector Store is nothing but a database that stores information in the form of numerical vectors so that an AI system can search for information based on meaning.

Imagine you have 1,000 PDF documents.

You want to find information related to:

"Artificial Intelligence memory systems."

A normal keyword search may look for the exact words.

But a vector store can find documents with similar meanings, even when they use different words.

For example:

- "Memory systems in AI."
- "How intelligent agents store information."
- "Long-term storage for autonomous systems."

These sentences are related in meaning.

A vector store helps identify such related information.

Technical explanation:

A Vector Store is a specialized data storage system that stores numerical vector representations, called embeddings, along with associated information or metadata.

It supports similarity-based retrieval by comparing the vector representation of a query with stored vectors.

3.2 What is an Embedding?

Simple explanation:

An embedding is nothing but converting text, images or other information into a list of numbers that represents its meaning or features.

For example:

Text:

"Apple is a fruit."

An embedding model converts it into a numerical vector such as:

```
[0.21, 0.45, 0.72, 0.18, ...]
```

Another sentence:

"Banana is a fruit."

It may produce a vector that is relatively close to the first vector because both sentences have related meanings.

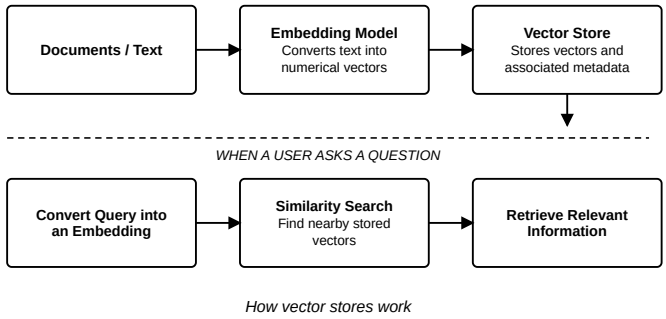
Technical explanation:

An embedding is a dense numerical representation of data in a multidimensional vector space.

Semantically similar inputs are generally represented by nearby vectors, allowing similarity-based search.

3.3 How Vector Stores Work

DIAGRAM



How vector stores work

3.4 Steps involved

1. Data collection: Collect documents, text or other information.
2. Chunking: Divide large documents into smaller sections.
3. Embedding generation: Convert each section into a numerical vector.
4. Storage: Store the vectors and related information in the vector database.
5. Query processing: Convert the user's question into an embedding.
6. Similarity search: Compare the query vector with stored vectors.
7. Retrieval: Return the most relevant information.

3.5 Similarity Search Methods

1. Cosine Similarity

Simple explanation: It measures how similar two vectors are based on the angle between them.

Technical explanation: Cosine similarity measures the cosine of the angle between two non-zero vectors.

Cosine Similarity = $(A \cdot B) / (\|A\| \|B\|)$

A value closer to 1 generally indicates greater similarity in direction.

2. Euclidean Distance

Simple explanation: It measures the straight-line distance between two vectors.

Technical explanation: Euclidean distance calculates the geometric distance between two points in vector space.

$d(A, B) = \sqrt{\Sigma (A_i - B_i)^2}$, $i = 1 \ldots n$

For this measure, smaller distances indicate closer vectors.

3. Dot Product

Simple explanation: It compares vectors using the sum of the products of their corresponding values.

Technical explanation: Dot-product similarity measures the inner product of two vectors and is commonly used in vector retrieval systems.

3.6 Popular Vector Databases

Pinecone

Managed vector database for similarity search and AI applications.

ChromaDB

Popular vector database for AI and RAG applications.

FAISS

Library developed by Meta for efficient similarity search and clustering.

Weaviate

Vector database supporting semantic search and AI applications.

3.7 Advantages

- Finds information based on meaning.
- Supports semantic search.
- Handles large collections of information.
- Useful for RAG applications.
- Helps retrieve relevant documents.

3.8 Applications

- AI chatbots.
- Document search.
- Recommendation systems.
- Image similarity search.
- Enterprise knowledge management.

4. RETRIEVAL-AUGMENTED GENERATION (RAG)

4.1 What is RAG?

Simple explanation:

RAG is nothing but allowing an AI model to search for relevant information from external documents before generating an answer.

Imagine you ask an AI assistant:

"What are the five important topics in my syllabus?"

Instead of depending only on what it already knows, the AI searches your syllabus PDF, retrieves the relevant information and generates the answer.

This is RAG.

Technical explanation:

Retrieval-Augmented Generation (RAG) is an architecture that combines information retrieval with language-model generation.

It retrieves relevant information from external knowledge sources and supplies it as context to a language model, which then generates a response grounded in the retrieved information.

4.2 Why is RAG required?

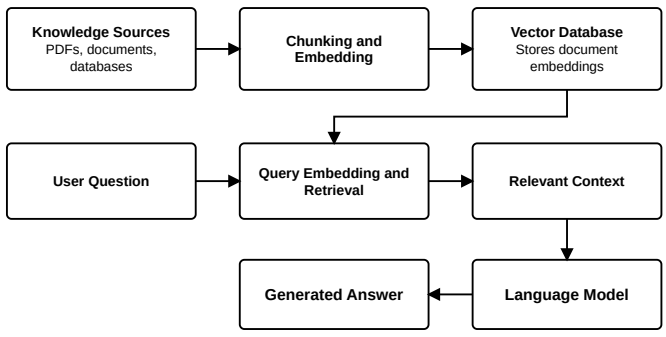
A language model may have limitations:

- It may not know the latest information.
- It may not know private organizational documents.
- It may generate incorrect information.
- It cannot automatically access every external database.
- It has a limited context window.

RAG helps by retrieving relevant information from external knowledge sources.

4.3 Architecture of RAG

DIAGRAM



Architecture of RAG

4.4 Working of RAG

Step 1: Document Collection

The system collects documents from PDFs, websites, databases or other sources.

Step 2: Document Chunking

Large documents are divided into smaller pieces.

Why?

Because searching an entire large document at once is inefficient. Smaller sections make it easier to retrieve the required information.

Step 3: Embedding Generation

Each document chunk is converted into a numerical vector using an embedding model.

Step 4: Vector Storage

The generated vectors are stored in a vector database.

Step 5: User Query

The user asks a question.

Step 6: Retrieval

The system converts the question into an embedding and searches for semantically similar document chunks.

Step 7: Generation

The retrieved information is provided to the language model as context.

Step 8: Final Answer

The language model generates a response using the retrieved information and its language capabilities.

4.5 Types of RAG

1. Naive RAG

Simple explanation: The system directly searches relevant information and gives it to the AI model.

Technical explanation: Naive RAG follows a basic retrieve-then-generate pipeline with relatively limited query processing and retrieval refinement.

2. Advanced RAG

Simple explanation: The system improves the searching process before and after retrieving information.

Technical explanation: Advanced RAG uses techniques such as query rewriting, reranking, hybrid retrieval, metadata filtering and improved document processing.

3. Modular RAG

Simple explanation: RAG is divided into separate components that can be customized.

Technical explanation: Modular RAG uses independent retrieval, transformation, routing and generation components that can be combined into more flexible pipelines.

4.6 Advantages

- Access to external knowledge.
- Can use updated information.
- Supports private document search.
- Improves factual grounding.
- Reduces dependence on the model's internal knowledge.

4.7 Limitations

- Incorrect retrieval can produce incorrect answers.
- Requires a suitable knowledge base.
- Poor document chunking affects retrieval quality.
- Embedding and retrieval operations require resources.
- The language model can still generate unsupported information.

5. GRAPHRAG

5.1 What is GraphRAG?

Simple explanation:

GraphRAG is nothing but an advanced form of RAG that uses relationships between pieces of information to answer questions.

Imagine you have a collection of documents about companies.

A normal RAG system may retrieve individual paragraphs.

But GraphRAG can also understand relationships such as:

- Company A owns Company B.
- Person X works for Company A.
- Company B manufactures Product C.

It uses these connections to answer questions involving multiple related entities.

Technical explanation:

GraphRAG is a retrieval-augmented generation approach that combines knowledge graphs with language models.

It extracts entities and relationships from documents, organizes them into a graph and uses graph-based retrieval and summarization to answer complex questions.

5.2 What is a Knowledge Graph?

Simple explanation:

A Knowledge Graph is nothing but information organized as connected entities.

For example:

- Ajay → studies → Computer Science.
- Ajay → learns → Agentic AI.
- Agentic AI → uses → Memory Systems.

These relationships form a graph.

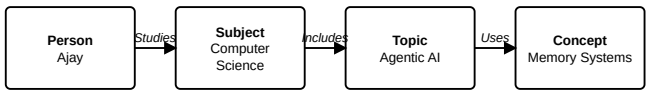
Technical explanation:

A knowledge graph represents information through nodes and edges.

- Nodes represent entities.
- Edges represent relationships between entities.

5.3 Knowledge Graph Example

DIAGRAM



Knowledge graph example

This representation makes relationships between entities easier to understand.

5.4 Working of GraphRAG

1. Document collection: Collect information from different documents.
2. Entity extraction: Identify important people, organizations, places and concepts.
3. Relationship extraction: Identify how the entities are connected.
4. Graph construction: Store the entities as nodes and relationships as edges.
5. Community detection: Identify groups of strongly related entities.
6. Graph retrieval: Retrieve relevant entities, relationships or community information.
7. Answer generation: Provide the retrieved information to the language model to generate an answer.

5.5 GraphRAG Architecture

Documents → Entity and Relationship Extraction → Knowledge Graph Construction → Graph Retrieval and Summarization → LLM Generates Answer

5.6 RAG vs GraphRAG

RAG	GraphRAG
Retrieves relevant document chunks.	Uses entities and relationships in a graph.
Primarily uses semantic similarity or other retrieval methods.	Uses graph-based retrieval and related methods.
Good for direct factual questions.	Useful for relationship-based and complex questions.
Often uses vector databases.	Uses knowledge graphs, often alongside vector retrieval.
Example: Find a definition in a PDF.	**Example:** Find connections between several companies.

5.7 Applications

- Enterprise knowledge management.
- Research analysis.
- Financial relationship analysis.
- Healthcare knowledge systems.
- Complex document question answering.

Important: GraphRAG is particularly useful when the answer requires understanding relationships across multiple documents rather than retrieving one relevant paragraph.

6. HIERARCHICAL MEMORY

6.1 What is Hierarchical Memory?

Simple explanation:

Hierarchical Memory is nothing but organizing an AI agent's memory into different levels, just like folders inside a computer.

Imagine your computer:

- Main folder: College.
- Subfolder: Subjects.
- Subfolder: Agentic AI.
- File: Module 2 notes.

You can store information in an organized structure and retrieve it whenever required.

An AI agent uses a similar approach to organize memory.

Technical explanation:

Hierarchical Memory is a memory architecture in which information is organized into multiple levels based on abstraction, importance, time scale or usage.

It allows agents to manage large amounts of information efficiently by separating immediate context from summarized knowledge and persistent information.

6.2 Levels of Hierarchical Memory

Level 1: Working Memory

Simple explanation: Information needed for the current task.

Example: The question currently being solved.

Level 2: Short-Term Memory

Simple explanation: Information needed during the current conversation or activity.

Example: Previous instructions in a conversation.

Level 3: Episodic Memory

Simple explanation: Records of past experiences and events.

Example: A previous attempt to solve a programming problem.

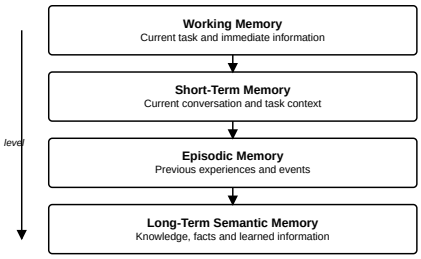
Level 4: Long-Term Semantic Memory

Simple explanation: Important knowledge and facts stored for future use.

Example: Knowledge about Python programming.

6.3 Architecture

DIAGRAM



Hierarchical memory architecture

This is one possible conceptual organization; actual agent architectures may arrange these memory types differently or use them in parallel.

6.4 Why is Hierarchical Memory needed?

- Manages large amounts of information.
- Organizes information according to importance.
- Supports efficient retrieval.
- Separates temporary and permanent information.
- Reduces unnecessary context usage.

6.5 Example

Imagine an AI personal assistant.

It stores:

Memory level	Stored information
Working	Current question about Python
Short-term	Current discussion about programming
Episodic	Previous Python debugging experience
Long-term semantic	Knowledge about Python syntax

When the user asks a new programming question, the agent retrieves the relevant information from the appropriate memory level.

7. MEMGPT ARCHITECTURE

7.1 What is MemGPT?

Simple explanation:

MemGPT is nothing but a memory-management approach that helps an AI agent manage more information than it can keep in its active context at one time.

Think about your computer.

Your computer has limited RAM.

When RAM becomes insufficient, the operating system uses storage to keep information and loads it into RAM when required.

MemGPT uses a similar idea for language-model memory.

Technical explanation:

MemGPT is an operating-system-inspired architecture for managing the limited context window of language models.

It separates active context from external memory and allows the agent to retrieve, store and update information through controlled memory operations.

7.2 Why is MemGPT needed?

Language models have a limited context window.

The context window determines how much information a model can process at one time.

For example, imagine an AI agent working on a project for several months.

It cannot necessarily keep every conversation, file and decision inside its active context.

MemGPT helps by managing information between active context and external memory.

7.3 Main Components

1. Main Context

Simple explanation: Information currently available to the AI model.

Technical explanation: The active context contains the current instructions, relevant conversation history and information required for immediate reasoning.

2. External Memory

Simple explanation: Information stored outside the active context.

Technical explanation: External memory stores additional information that can be retrieved when required.

3. Memory Management

Simple explanation: Decides which information should stay active and which should be stored externally.

Technical explanation: Memory management controls information transfer between the active context and external memory.

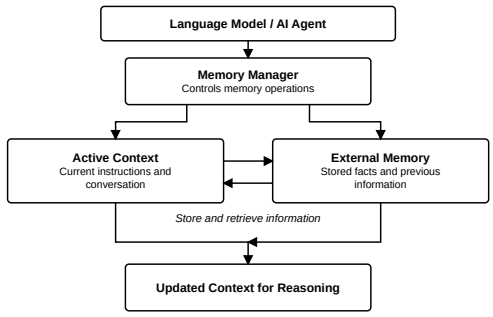
4. Retrieval Mechanism

Simple explanation: Finds previously stored information.

Technical explanation: Retrieval identifies and loads relevant external information into the active context.

7.4 MemGPT Architecture

DIAGRAM



MemGPT architecture

7.5 Working of MemGPT

1. The user provides a request.
2. The agent checks its active context.
3. If the required information is available, it uses it.
4. If information is missing, the agent retrieves it from external memory.
5. The retrieved information is loaded into the active context.
6. The agent performs the required reasoning.
7. Important new information can be saved to external memory.
8. The process continues for future interactions.

7.6 Advantages

- Supports long conversations.
- Manages limited context windows.
- Allows persistent information storage.
- Improves information retrieval.
- Supports long-running tasks.

7.7 Applications

- Personal AI assistants.
- Long-term research agents.
- Project management agents.
- Conversational AI systems.
- Autonomous software agents.

Important comparison: MemGPT is an architecture for managing memory, whereas RAG is primarily a retrieval-and-generation approach for using external knowledge. They can be combined.

8. CONTEXT MANAGEMENT

8.1 What is Context Management?

Simple explanation:

Context Management is nothing but deciding what information an AI model should remember and use while answering a question.

Imagine you are studying a 500-page textbook.

You cannot keep every page in front of you at the same time.

Instead, you select the pages related to the topic you are studying.

AI models also need to select relevant information because their context window is limited.

Technical explanation:

Context Management refers to the process of organizing, selecting, prioritizing, storing and updating information supplied to a language model during inference.

It ensures that relevant instructions, conversation history, retrieved information and tool outputs are available within the model's context window.

8.2 What is a Context Window?

Simple explanation:

A context window is nothing but the maximum amount of information an AI model can process in one interaction.

It includes:

- User messages.
- System instructions.
- Previous conversation.
- Retrieved documents.
- Tool results.
- Model-generated content.

Technical explanation:

A context window is the maximum number of tokens that a language model can process as input and maintain in its active context for a given inference operation.

A token may represent a word, part of a word or another text unit.

8.3 Why is Context Management needed?

Suppose an AI agent is working with a very long conversation.

If all previous messages are added repeatedly, the context may become too large.

Context management helps by:

1. Removing irrelevant information.
2. Summarizing older conversations.
3. Keeping important instructions.
4. Retrieving relevant memories.
5. Managing tool outputs.
6. Prioritizing important information.

8.4 Context Management Techniques

1. Context Truncation

Simple explanation: Remove older or unnecessary information to make space.

Technical explanation: Context truncation reduces the active context by discarding selected tokens or messages.

2. Summarization

Simple explanation: Replace a long conversation with a shorter explanation containing the important points.

Technical explanation: Summarization compresses previous interactions into a concise representation that preserves relevant information.

3. Retrieval

Simple explanation: Search external memory and bring only the required information into the current context.

Technical explanation: Retrieval-based context management selects relevant external information and injects it into the active context.

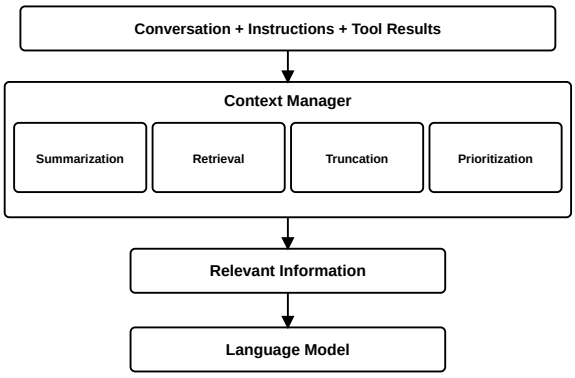
4. Prioritization

Simple explanation: Keep important information and give less attention to unimportant information.

Technical explanation: Context prioritization ranks information based on relevance, importance, recency or task requirements.

8.5 Architecture

DIAGRAM



Context management architecture

8.6 Advantages

- Prevents unnecessary context usage.
- Improves relevance.
- Supports long conversations.
- Reduces repeated information.
- Helps manage limited context windows.

9. PERSISTENT MEMORY

9.1 What is Persistent Memory?

Simple explanation:

Persistent Memory is nothing but the ability of an AI agent to remember information even after a conversation or program has ended.

For example:

Today, you tell an AI assistant that you prefer Python.

Tomorrow, you open a new conversation.

If the assistant has stored this preference in persistent memory, it can use that information again.

Technical explanation:

Persistent Memory is a memory mechanism that stores information in durable storage so that it remains available across sessions, restarts or separate tasks.

It allows AI agents to retain user preferences, important facts, knowledge and previous task information.

9.2 Why is Persistent Memory required?

Without persistent memory, an agent may lose information when a session ends.

Persistent memory helps the agent:

- Remember previous interactions.
- Store user preferences.
- Maintain project information.
- Continue long-running tasks.
- Retrieve historical information.

9.3 Working

User Interaction → Extract Important Information → Persistent Storage — Database, files or memory store → **FUTURE SESSION → Retrieve Stored Information → Use Previous Information**

9.4 Types of Persistent Storage

1. File-Based Storage

Information is saved in files such as TXT, JSON or CSV.

2. Database Storage

Information is saved in databases such as MySQL, PostgreSQL or MongoDB.

3. Vector Database Storage

Information is stored as embeddings to support semantic retrieval.

4. Knowledge Graph Storage

Information is stored as entities and relationships.

9.5 Python Example

```python
import json

# Store information
memory = {
    "name": "Ajay",
    "subject": "Agentic AI",
    "language": "Python"
}

with open("memory.json", "w") as file:
    json.dump(memory, file)

# Retrieve information
with open("memory.json", "r") as file:
    saved_memory = json.load(file)

print(saved_memory["subject"])
```

Output:

```
Agentic AI
```

Unlike an ordinary Python dictionary, the JSON file remains available after the program ends.

9.6 Advantages

- Retains information across sessions.
- Supports personalization.
- Helps continue previous tasks.
- Reduces repeated instructions.
- Enables long-term agent workflows.

Important: Persistent memory describes how long information is retained. Episodic memory describes the type of information being remembered. An episodic memory can be stored persistently.

10. EPISODIC MEMORY

10.1 What is Episodic Memory?

Simple explanation:

Episodic Memory is nothing but remembering previous experiences and events.

Imagine you attempted a programming problem yesterday.

- You wrote a program.
- It produced an error.
- You corrected the error.
- Finally, the program worked.

If an AI agent remembers this complete experience, it can use it to solve similar problems in the future.

This is episodic memory.

Technical explanation:

Episodic Memory is a type of memory that stores records of specific events, interactions, observations and experiences associated with their context.

It may include details such as the task, actions performed, results obtained, timestamps and outcomes.

10.2 What does Episodic Memory store?

An episodic memory may contain:

- What happened.
- When it happened.
- Where it happened.
- Which actions were performed.
- What result was obtained.
- What the agent learned from the experience.

10.3 Example

An AI coding agent performs a debugging task.

It stores:

```
Task:
Fix a Python program.

Problem:
NameError occurred.

Action:
Corrected the variable name.
```

```
Result:
Program executed successfully.

Experience:
Check variable names during debugging.
```

Later, when the agent encounters a similar error, it can retrieve this previous experience.

10.4 Working of Episodic Memory

Agent Performs a Task → Observe Actions and Results → Store Experience → Retrieve Similar Past Experience → Improve Future Decision

10.5 Episodic Memory vs Semantic Memory

Episodic Memory	Semantic Memory
Stores personal or agent experiences.	Stores general facts and knowledge.
Includes events and outcomes.	Includes concepts and information.
Often contains time and context.	Usually stores context-independent knowledge.
Example: Previous debugging experience.	**Example:** Python is a programming language.

10.6 Applications

- Autonomous robotics.
- Personal AI assistants.
- Software debugging agents.
- Long-running research agents.
- Experience-based decision-making.

11. EXPERIENCE REPLAY

11.1 What is Experience Replay?

Simple explanation:

Experience Replay is nothing but storing previous experiences and using them again to improve future decisions.

Imagine you are learning to play a game.

Whenever you make a move, you observe whether it was successful or unsuccessful.

Instead of forgetting every previous move, you store those experiences and review them later to improve your performance.

AI agents use a similar method.

Technical explanation:

Experience Replay is a learning technique, commonly used in reinforcement learning, in which an agent stores previous interactions with an environment and samples them later to improve its learning process.

The stored experiences are commonly represented as:

Experience = (s_t, a_t, r_t, s_{t+1})

Where:

Symbol	Meaning
s_t	Current state
a_t	Action performed
r_t	Reward received
s_{t+1}	Next state

11.2 Why is Experience Replay needed?

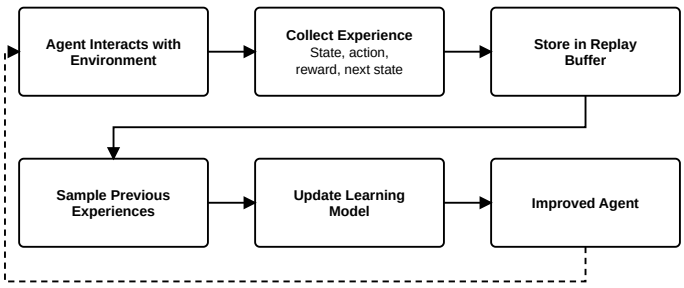
Without experience replay, an agent may learn only from its latest interaction.

This can make learning unstable and cause it to forget useful earlier experiences.

Experience replay allows the agent to learn from previously stored experiences.

11.3 Working of Experience Replay

DIAGRAM



Working of experience replay

11.4 What is a Replay Buffer?

Simple explanation:

A Replay Buffer is nothing but a storage area where the agent saves its previous experiences.

Technical explanation:

A replay buffer is a data structure that stores previously collected experience transitions for later sampling during model training.

It may use a fixed capacity, replacing older experiences when the buffer becomes full.

11.5 Types of Experience Replay

1. Uniform Experience Replay

Simple explanation: The agent randomly selects previous experiences from memory.

Technical explanation: Uniform replay samples stored transitions with approximately equal probability.

2. Prioritized Experience Replay

Simple explanation: The agent gives more importance to experiences that contain larger learning errors.

Technical explanation: Prioritized Experience Replay samples transitions according to their priority, often based on the magnitude of the temporal-difference error.

This helps the agent focus more on informative experiences.

11.6 Python Example

A simple demonstration of storing and replaying experiences:

```python
import random

replay_buffer = []

# Store experiences
replay_buffer.append(("A", "Right", 10, "B"))
replay_buffer.append(("B", "Left", -5, "A"))
replay_buffer.append(("B", "Right", 20, "C"))

# Replay a random experience
experience = random.choice(replay_buffer)
print("Replayed experience:", experience)
```

Possible output:

```
Replayed experience: ('B', 'Right', 20, 'C')
```

This demonstrates experience storage and sampling. A complete reinforcement-learning implementation would use the sampled experiences to update a learning model.

11.7 Advantages

- Reuses previous experiences.
- Improves learning efficiency.
- Reduces correlation between consecutive training samples when randomly sampled.
- Supports stable reinforcement learning.
- Helps learn from past mistakes.

11.8 Applications

- Game-playing agents.
- Robotics.
- Autonomous navigation.
- Reinforcement learning.
- Decision-making systems.

12. VOYAGER PATTERN

12.1 What is Voyager?

Simple explanation:

Voyager is nothing but an AI agent that continuously learns new skills, stores them and reuses those skills to solve future problems.

Imagine a student learning programming.

Today, the student learns how to write a loop.

Tomorrow, the student learns how to use functions.

Later, when solving a bigger problem, the student combines the previously learned skills.

Voyager follows a similar concept, but in an autonomous AI environment.

Technical explanation:

Voyager is an autonomous lifelong-learning agent developed for the Minecraft environment.

It uses a large language model to generate executable code, an automatic curriculum to propose increasingly challenging tasks and a skill library to store and reuse successful programs.

It improves its capabilities through repeated interaction with the environment.

12.2 Why is Voyager important?

Traditional AI agents may solve one task and then start another task without effectively reusing previous solutions.

Voyager addresses this limitation by maintaining a library of previously learned skills.

It can reuse existing skills and build more advanced skills over time.

12.3 Three Main Components of Voyager

1. Automatic Curriculum

Simple explanation:

The agent decides what it should learn next.

For example, first it learns basic movements, then collecting resources and later crafting objects.

Technical explanation:

Automatic Curriculum generates increasingly challenging tasks based on the agent's current capabilities and progress.

It encourages continuous skill acquisition.

2. Skill Library

Simple explanation:

The agent stores previously learned skills so that it can use them again.

For example:

- Move forward.
- Collect wood.
- Craft tools.
- Build structures.

Technical explanation:

The Skill Library stores reusable executable code for previously acquired skills.

It enables the agent to retrieve and combine existing capabilities when solving new tasks.

3. Iterative Prompting Mechanism

Simple explanation:

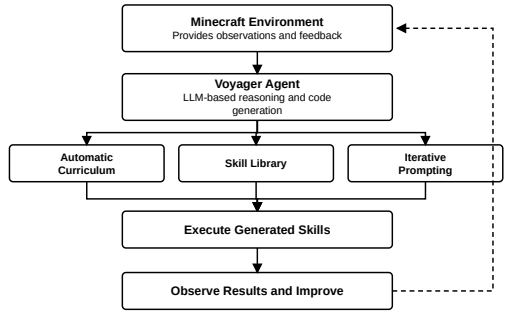
The agent repeatedly tries a task, checks the result and improves its code when something goes wrong.

Technical explanation:

The iterative prompting mechanism uses environmental feedback and execution errors to revise generated code and improve task performance.

12.4 Voyager Architecture

DIAGRAM



Voyager architecture

12.5 Working of Voyager

Step 1: Observe the Environment

The agent receives information about its current Minecraft environment.

Step 2: Select a Task

The automatic curriculum proposes a suitable task.

Example: Collect wood.

Step 3: Generate Code

The language model generates a program to perform the task.

Step 4: Execute the Program

The agent executes the generated code in the Minecraft environment.

Step 5: Evaluate the Result

The agent checks whether the task was completed successfully.

Step 6: Store the Skill

If successful, the program is stored in the skill library for future reuse.

Step 7: Learn More Complex Skills

The agent uses previously learned skills to perform more challenging tasks.

12.6 Example

Imagine Voyager learning Minecraft.

Stage	Skill learned
1	Move around the environment
2	Collect wood
3	Craft basic tools
4	Mine resources
5	Build structures

The agent can reuse earlier skills while learning more advanced tasks.

12.7 Advantages

- Supports lifelong learning.
- Reuses previously learned skills.
- Reduces repeated work.
- Supports automatic curriculum generation.
- Improves performance through environmental feedback.
- Can combine simple skills into complex behaviors.

12.8 Limitations

- Depends on the quality of generated code.
- Requires a suitable environment.
- Incorrectly learned skills can affect future tasks.
- Complex tasks may require significant computation.

Important exam point: Voyager combines Automatic Curriculum, Skill Library and Iterative Prompting for lifelong learning in Minecraft.

13. IMPORTANT COMPARISONS

These comparisons are particularly useful for 2-mark and 10-mark questions.

13.1 RAG vs GraphRAG

RAG	GraphRAG
Retrieves relevant document chunks.	Uses graph entities and relationships.
Uses semantic retrieval and other search methods.	Uses graph-based retrieval and summaries.
Suitable for direct information retrieval.	Suitable for relationship-based questions.
Often uses vector databases.	Uses knowledge graphs and may combine vector search.

13.2 Episodic Memory vs Persistent Memory

Episodic Memory	Persistent Memory
Stores past events and experiences.	Stores information across sessions.
Focuses on what happened.	Focuses on retaining information over time.
Includes context and outcomes.	Can store facts, preferences and experiences.
Example: Previous debugging experience.	**Example:** Saving user information in a database.

13.3 MemGPT vs RAG

MemGPT	RAG
Manages information beyond active context.	Retrieves external information to generate answers.
Uses active context and external memory.	Uses a retrieval and generation pipeline.
Supports long-running conversations.	Supports knowledge-grounded responses.
Example: Retrieving previous project discussions.	**Example:** Searching a PDF for an answer.

13.4 Context Management vs Memory Management

Context Management	Memory Management
Organizes information in the active context.	Organizes information across memory storage.
Works with the context window.	Works with short-term and long-term storage.
Uses summarization and prioritization.	Uses storage, retrieval and updating.
Example: Removing irrelevant conversation history.	**Example:** Saving previous project details.

13.5 Experience Replay vs Episodic Memory

Experience Replay	Episodic Memory
Reuses previous experiences for learning.	Stores records of previous events.
Commonly used in reinforcement learning.	Used in many types of AI agents.
Samples stored experiences.	Retrieves relevant past experiences.
Example: Training from previous game moves.	**Example:** Remembering a previous task and its result.

14. IMPORTANT 2-MARK QUESTIONS

Prepare these definitions for short-answer questions.

1. Define Test-Time Compute.

Test-Time Compute refers to the additional computational resources used during inference to improve an AI model's reasoning, verification and problem-solving capabilities.

2. What is Generate–Verify–Refine?

Generate–Verify–Refine is an iterative paradigm in which an AI system generates a solution, verifies its correctness and refines it based on feedback.

3. Define Vector Store.

A Vector Store is a database that stores numerical vector representations of information and supports similarity-based retrieval.

4. What is an embedding?

An embedding is a numerical representation of data in a vector space that captures its features or semantic meaning.

5. What is cosine similarity?

Cosine similarity measures the similarity between two vectors based on the cosine of the angle between them.

6. Define RAG.

Retrieval-Augmented Generation is an AI technique that retrieves relevant external information and provides it as context to a language model for generating answers.

7. What is GraphRAG?

GraphRAG combines knowledge graphs and language models to retrieve information using entities and their relationships.

8. What is a Knowledge Graph?

A Knowledge Graph is a structured representation of information using nodes to represent entities and edges to represent relationships.

9. Define Hierarchical Memory.

Hierarchical Memory organizes an AI agent's information into multiple levels based on importance, abstraction or usage.

10. What is MemGPT?

MemGPT is an operating-system-inspired architecture that manages a language model's limited context using active context and external memory.

11. Define Context Management.

Context Management is the process of selecting, organizing and prioritizing information within a language model's context window.

12. What is a Context Window?

A context window is the maximum amount of information, measured in tokens, that a language model can process in a given inference operation.

13. Define Persistent Memory.

Persistent Memory stores information in durable storage so that an AI agent can retrieve it across different sessions.

14. What is Episodic Memory?

Episodic Memory stores records of specific events, interactions and experiences of an AI agent.

15. What is Experience Replay?

Experience Replay is a reinforcement-learning technique in which an agent stores previous experiences and samples them later to improve learning.

16. What is a Replay Buffer?

A Replay Buffer is a storage structure that holds previous experience transitions for later sampling during training.

17. Expand Voyager.

Voyager is the name of an autonomous lifelong-learning agent developed for Minecraft. It uses automatic curriculum, a skill library and iterative prompting.

18. What is Automatic Curriculum in Voyager?

Automatic Curriculum is a mechanism that generates progressively challenging tasks based on the agent's current capabilities.

19. What is a Skill Library?

A Skill Library stores reusable executable programs or skills that an agent can retrieve and use for future tasks.

20. What is Iterative Prompting?

Iterative Prompting is a process in which an AI model repeatedly generates and improves its output using feedback from previous attempts.